AprilTag Id Sorting

1. Function Description

After the program starts, the camera recognizes the AprilTag and will frame the AprilTag in the image and print the corresponding ID value. Press the spacebar to start the sorting program. The robotic arm will grip the recognized AprilTag block and place the AprilTag at the specified position based on the recognized ID.

2. Startup and Operation

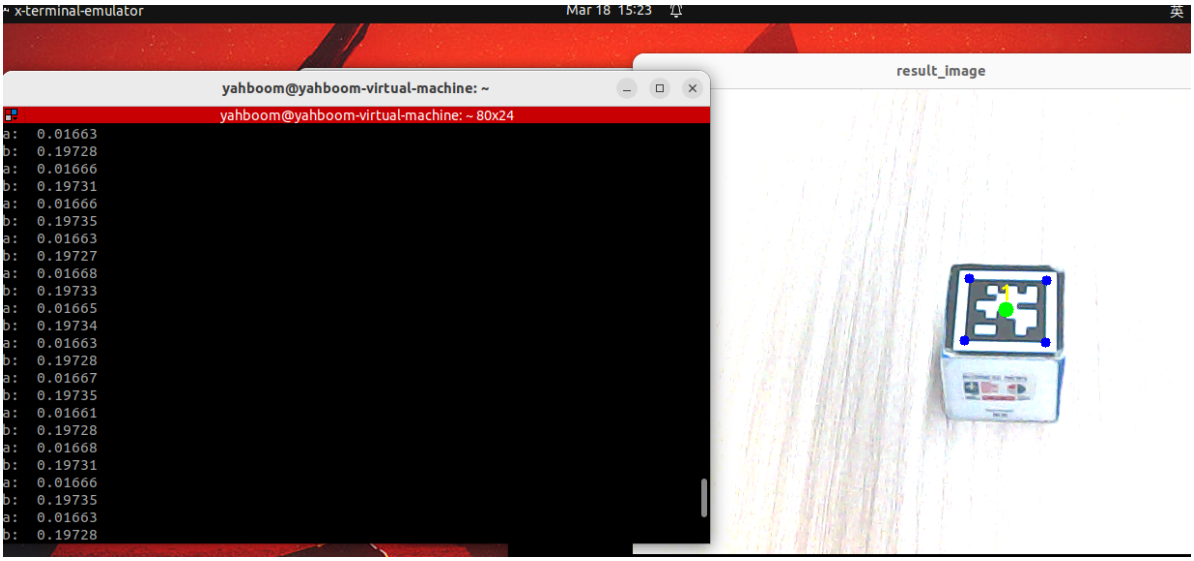
2.1 Startup Commands

After opening the terminal, enter the following commands:

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# AprilTag recognition
ros2 run dofbot_sorting_3d apriltag_sorting
# Gripping
ros2 run dofbot_sorting_3d grasp
```

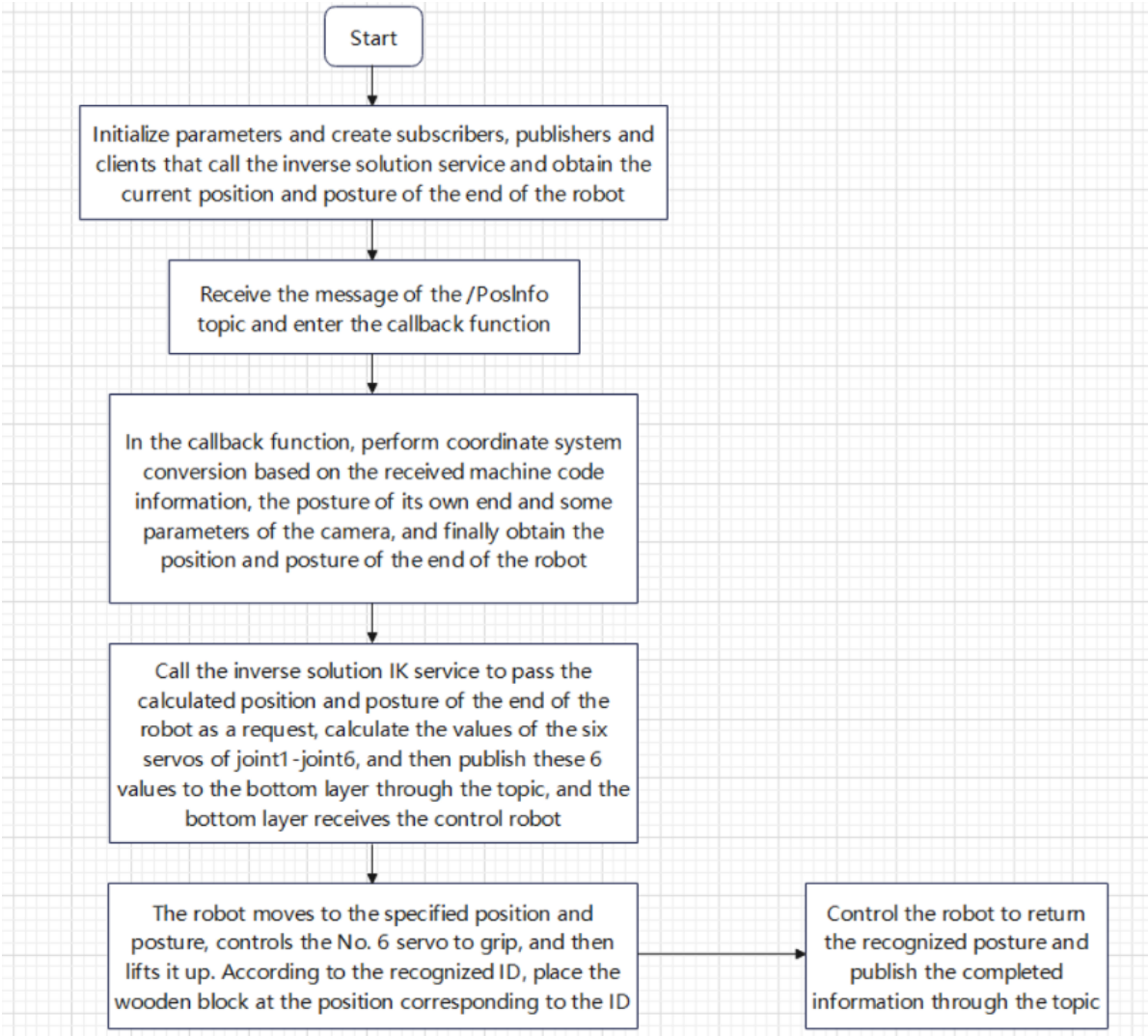
2.2 Operation

Use a **3x3 square wooden block**, click on the image frame with the mouse, then press the spacebar on the keyboard. The robotic arm performs a series of calculations based on the center point coordinates of the recognized AprilTag block and the xy coordinates of the center point. Then, according to the calculation results, it will lower the gripper to grip the recognized AprilTag block. After gripping is complete, it will place the AprilTag at the set position based on the ID value of the gripped AprilTag. After placement is complete, it will return to the recognition pose to recognize the next AprilTag. One terminal prints the center value of the recognized AprilTag and the depth information of that center value, while the other prints the real-time calculation process.



3. Program Flowchart

grasp.py



4. Core Code Analysis

apriltag_sorting.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/apriltag_sorting
.py
```

Import necessary libraries

```
import rclpy
from rclpy.node import Node
import cv2
...
from dofbot_driver.compute_joint5 import *
```

Initialize program parameters, create publishers and subscribers

```
def __init__(self):
    super().__init__('apriltag_detect')
```

```
    self.Arm = Arm_Device()
    # Publish the initial pose of the robotic arm, which is also the recognition
pose
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]
    self.joint5 = Int16()
    self.detect_flag = False
    self.compute_height = True
    self.index = None
    self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')
    self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)
    # Create publisher for AprilTag information
    self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)
    self.subscription =
self.create_subscription(Bool,'grasp_done',self.GraspStatusCallback,qos_profile=
10)
    self.rgb_bridge = CvBridge()
    self.depth_bridge = CvBridge()
    # Flag to publish AprilTag information, when True, publish /TagInfo topic
data
    self.pubPos_flag = False
    self.pr_time = time.time()
    # Create AprilTag detector object, set some parameters as follows:
    '''
    searchpath: Specifies the path to search for tag models.
    families: Set the tag family to use, e.g., 'tag36h11'.
    nthreads: Number of parallel processing threads to improve detection speed.
    quad_decimate: Reduce input image resolution to reduce computation.
    quad_sigma: Standard deviation for Gaussian blur, affecting image
preprocessing.
    refine_edges: Whether to refine edges to improve detection accuracy.
    decode_sharpening: Sharpening parameter during decoding to enhance tag
contrast.
    debug: Debug mode switch for viewing information during detection
    '''
    self.at_detector = Detector(searchpath=['apriltags'],
                                families='tag36h11',
                                nthreads=8,
                                quad_decimate=2.0,
                                quad_sigma=0.0,
                                refine_edges=1,
                                decode_sharpening=0.25,
                                debug=0)
    self.target_id = 31
    self.Center_x_list = []
    self.Center_y_list = []
    self.heigh = 0.0
    self.CurEndPos = [-0.006,0.116261662208,0.0911289015753,-1.04719,-0.0,0.0]
    self.camera_info_K = [1021.16691, 0.0, 238.3799, 0.0, 1014.00486, 261.07339,
0.0, 0.0, 1.0]
    self.EndToCamMat = np.array([[-0.001 ,-1.000 ,-0.001 , 0.000],
                                 [-0.002  ,0.001 ,-1.000 ,-0.056],
                                 [ 1.000 ,-0.001 ,-0.002 ,-0.097],

[0.00000000e+00,0.00000000e+00,0.00000000e+00,1.00000000e+00]])
    self.get_current_end_pos()
    self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
    self.dist = 0.13
```

```
    # Subscribe to color image topic
    self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Grasp status callback function

```
def GraspStatusCallback(self,msg):
    if msg.data == True:
        print("Receive grasp_done msg: {}".format(msg.data))
        self.detect_flag = False  # Reset detection flag to allow next detection
        self.compute_height = True  # Reset height calculation flag
        self.pubPos_flag = False   # Reset publish flag

        # Add extra delay to ensure message processing is complete
        time.sleep(0.1)
```

grasp.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/grasp.py
```

Import necessary libraries

```
import rclpy
from rclpy.node import Node
...
# Import transforms3d library for handling transformations in 3D space, execute
conversions between quaternions, rotation matrices and Euler angles, support for
3D geometric operations and coordinate transformations
import transforms3d as tfs
# Import transformations for processing and calculating transformations in 3D
space, including conversions between quaternions and Euler angles
import tf_transformations as tf
import threading
from ament_index_python import get_package_share_directory
import yaml
import os
from Arm_Lib import Arm_Device
```

Initialize program parameters, create publishers, subscribers and clients

```
def __init__(self):
    # Initialize node / Initialize node
    super().__init__('color_grap')
    # Create arm device object / Create arm device object
    self.Arm = Arm_Device()
    # Create AprilTag position info subscriber / Create AprilTag position info
subscriber
    self.sub =
self.create_subscription(AprilTagInfo,'PosInfo',self.pos_callback,1)
    # Create joint5 angle subscriber / Create joint5 angle subscriber
    self.sub_joint5 =
self.create_subscription(Int16,"set_joint5",self.get_joint5_callback,1)
```

```
    # Create joint6 angle subscriber / Create joint6 angle subscriber
    self.sub_joint6 =
self.create_subscription(Int16,"set_joint6",self.get_joint6_callback,1)
    # Create grasp completion status publisher / Create grasp completion status
publisher
    self.pubGraspStatus = self.create_publisher(Bool, 'grasp_done',1)
    # Create kinematics service client / Create kinematics service client
    self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')
    # Grasp flag / Grasp flag
    self.grasp_flag = True
    # Initial joint angles / Initial joint angles
    self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 30.0]
    # Down joint angles / Down joint angles
    self.down_joint = [10.0, 70.0, 34.0, 16.0, 90.0,140.0]
    # Gripper joint angle / Gripper joint angle
    self.gripper_joint = 90.0
    # Current tag ID / Current tag ID
    self.cur_tagId = 0
    # Joint5 current angle / Joint5 current angle
    self.joint_5 = 90
    # Joint5 set angle / Joint5 set angle
    self.set_joint5 = 90
    # Joint6 current angle / Joint6 current angle
    self.joint_6 = 140
    # Initialization done / Initialization done
    print("Init Done")

    # Call inverse kinematics service, which calls the ik service content, assign
values to the required request parameters
    def grasp(self,pos_x,pos_y,pos_z):
        print("-------------------------------------------------")
        request = Kinemarics.Request()
        request.tar_x = pos_x
        request.tar_y = pos_y
        request.tar_z = pos_z + 0.03
        request.kin_name = "ik"
        # Target pitch value of the robotic arm end, unit is radians, this is
the current pitch value of the robotic arm end
        request.pitch = 1.04
        try:
            future = self.client.call_async(request)
            rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
            response = future.result()
            print("calcutelate_response: ",response)
            joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
            joints[0] = response.joint1 #response.joint1
            joints[1] = 180 - response.joint2
            joints[2] = 180 - response.joint3
            joints[3] = 180 - response.joint4
            print("self.joint_5: ",self.joint_5)
            target_joint5 = self.joint_5
            if target_joint5>0 and target_joint5 < 45:
                joint5 = 90 - target_joint5
            elif target_joint5>45 and target_joint5<135:
                joint5 = target_joint5
            elif target_joint5>=135:
                joint5 = target_joint5 - 90
            elif target_joint5>-45 and target_joint5<0:
```

```
                joint5 = 90 + abs(target_joint5)
            elif target_joint5<-45 and target_joint5>-135:
                joint5 = abs(target_joint5)
            elif target_joint5<-135:
                joint5 = abs(target_joint5) - 90
            self.set_joint5 = joint5
            joints[5] = 30

 self.Arm.Arm_serial_servo_write6(joints[0],joints[1],joints[2],joints[3],90,joi
nts[5],2000)
            time.sleep(2.0)
            self.move()

        except Exception:
           pass
```

Publish robotic arm target angle function pubTargetArm

```
def pubTargetArm(self, joints, id=6, angle=180.0, runtime=2000):
        print(joints)
 self.Arm.Arm_serial_servo_write6(joints[0],joints[1],joints[2],joints[3],joints
[4],joints[5],2000)
```

Grip and place function move

```
# According to the id value, change self.down_joint, this value represents the
position where the AprilTag block is placed
def move(self):
   # self.Arm.Arm_serial_servo_write(5, self.set_joint5, 1000)
    self.Arm.Arm_serial_servo_write(6, 140, 2000)
    time.sleep(2.5)
    #self.Arm.Arm_serial_servo_write(6, self.joint_6, 1000)
    #time.sleep(1.0)
    self.Arm.Arm_serial_servo_write(2, 120, 2000)
    time.sleep(2.5)
    print("self.cur_tagId",self.cur_tagId)
    if self.cur_tagId == 1:
        self.down_joint = p_1
        # #Blue position Blue position
        print("Setting to BLUE position")
    elif self.cur_tagId == 2:
        self.down_joint = p_2
        # #Green position Green position
        print("Setting to GREEN position")
    elif self.cur_tagId == 3:
        self.down_joint = p_3
        # #Red position Red position
        print("Setting to RED position")
    elif self.cur_tagId == 4:
        self.down_joint = p_4
        # #Yellow position Yellow position
        print("Setting to YELLOW position")
    else:
        # Default position or handling for other IDs
        self.down_joint = [155.0, 35, 70, 5, 60.0,140]
        print("Setting to DEFAULT position")
```

```
    print("self.down_joint",self.down_joint)
    self.down_joint[5] = self.joint_6
    self.Arm.Arm_serial_servo_write6_array(self.down_joint,1000)
    time.sleep(1)
    self.Arm.Arm_serial_servo_write(6, 90, 300)
    time.sleep(.3)
    self.Arm.Arm_serial_servo_write(2, 90, 1000)
    time.sleep(1)
    #Wait for the robotic arm to return to the initial position, then publish the
grasp completion message, change the grasp flag value to allow the gripper to
grip next time when conditions are met
    self.grasp_flag = True
    grasp_done = Bool()
    grasp_done.data = True
    self.Arm.Arm_serial_servo_write6_array(self.init_joints,1500)
    time.sleep(1.5)
    self.pubGraspStatus.publish(grasp_done)
```